

Ensemble learning

Yongmei Tan
ymtan@bupt.edu.cn

集成学习(Ensemble learning)

- 集成学习通过构建并结合多个学习器来完成学习任务

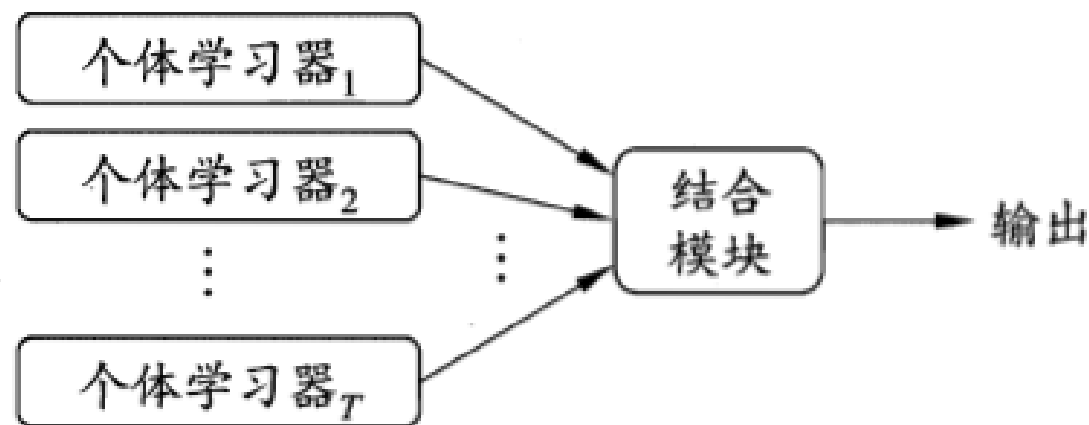


图 8.1 集成学习示意图

- 同质(homogeneous)集成：集成中只包含同种类型的“个体学习器”相应的学习算法称为“基学习算法(base learning algorithm)”
- 异质(heterogeneous)集成：个体学习器由不同的学习算法生成不存在的“基学习算法”

Why Ensemble

集成的泛化性能通常显著优于单个学习器的泛化性能

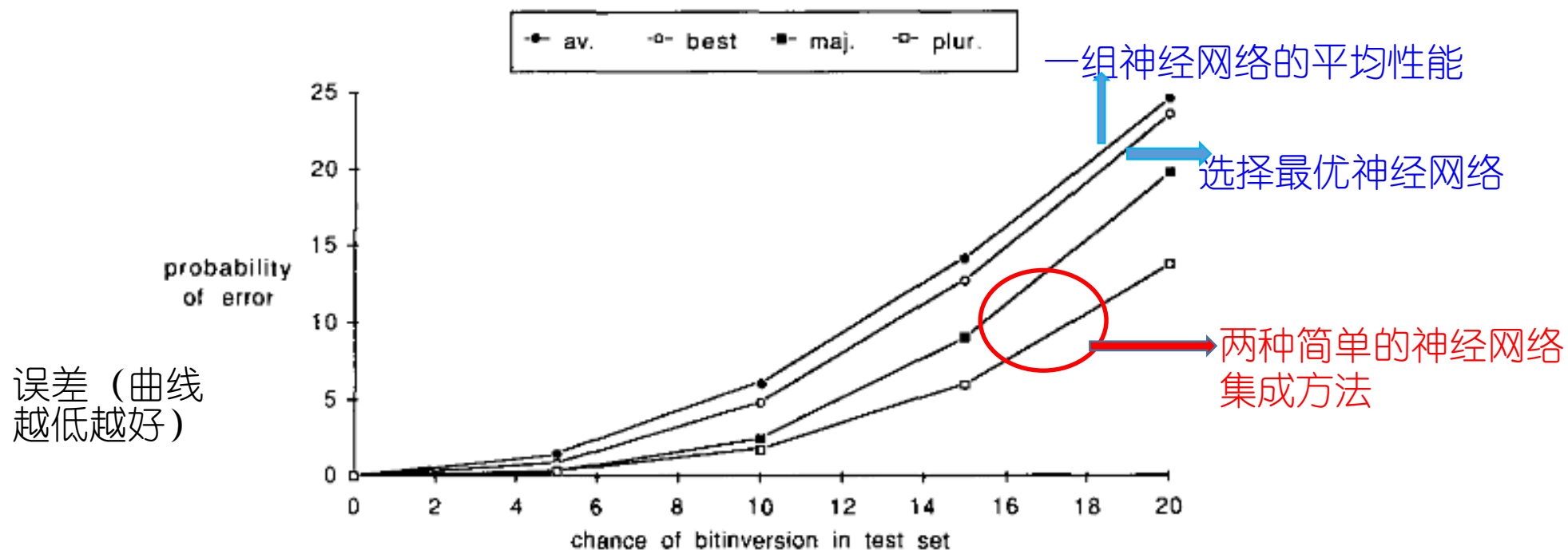


Fig. 4. Performance versus noise level in the test set is shown for individual and for consensus decisions. Data displayed shows the average and the best network, as well as collective decisions using majority and plurality for seven networks trained on individual training sets.

[hansen & Salamon, TPAMI90]

如何得到好的集成

- 个体学习器应“好而不同”

测试例1 测试例2 测试例3				测试例1 测试例2 测试例3				测试例1 测试例2 测试例3			
h_1	✓	✓	✗	h_1	✓	✓	✗	h_1	✓	✗	✗
h_2	✗	✓	✓	h_2	✓	✓	✗	h_2	✗	✓	✗
h_3	✓	✗	✓	h_3	✓	✓	✗	h_3	✗	✗	✓
集成	✓	✓	✓	集成	✓	✓	✗	集成	✗	✗	✗
(a) 集成提升性能				(b) 集成不起作用				(c) 集成起负作用			

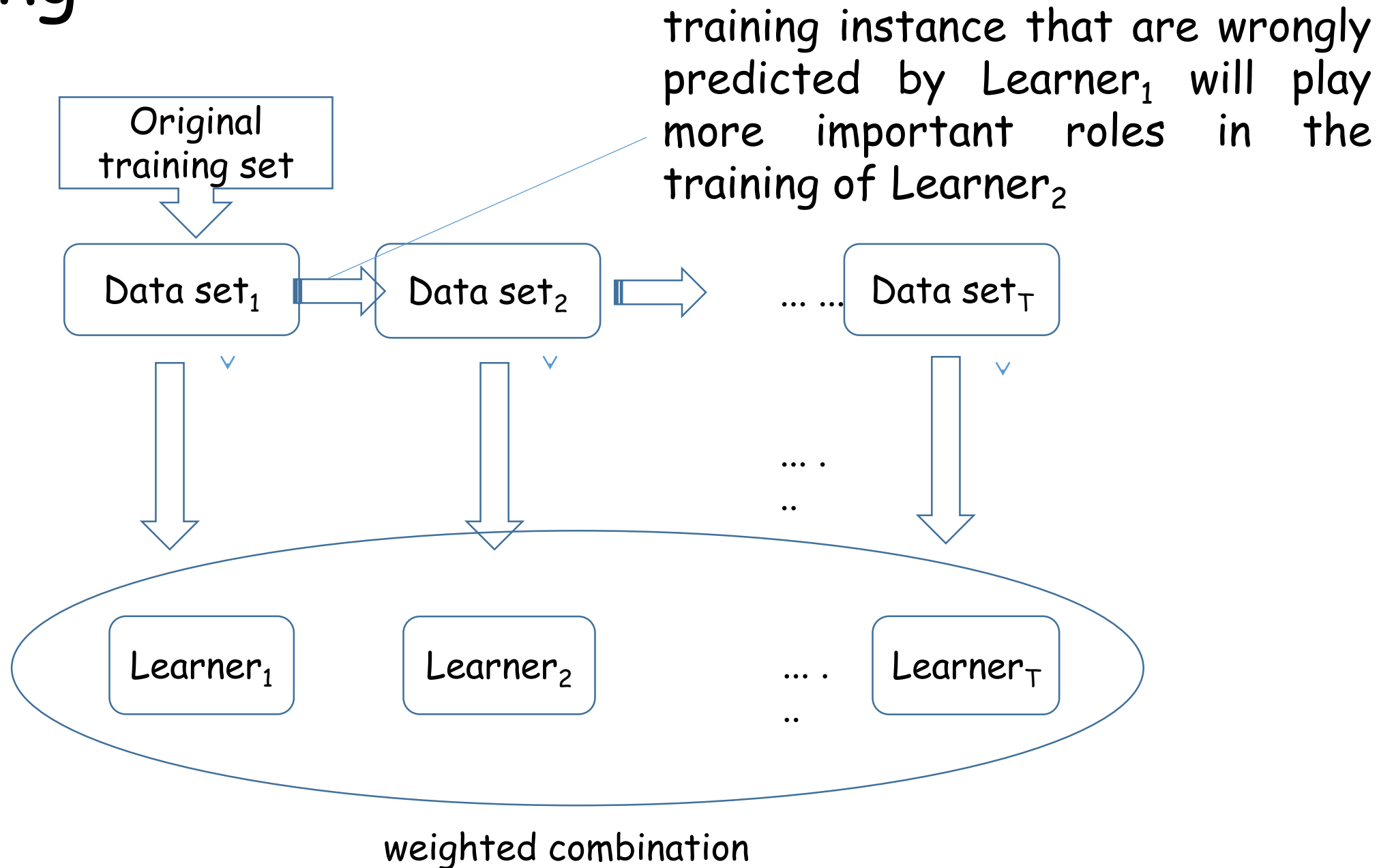
图 8.2 集成个体应“好而不同” (h_i 表示第 i 个分类器)

个体学习器要有一定的“准确性”，并且要有“多样性”。

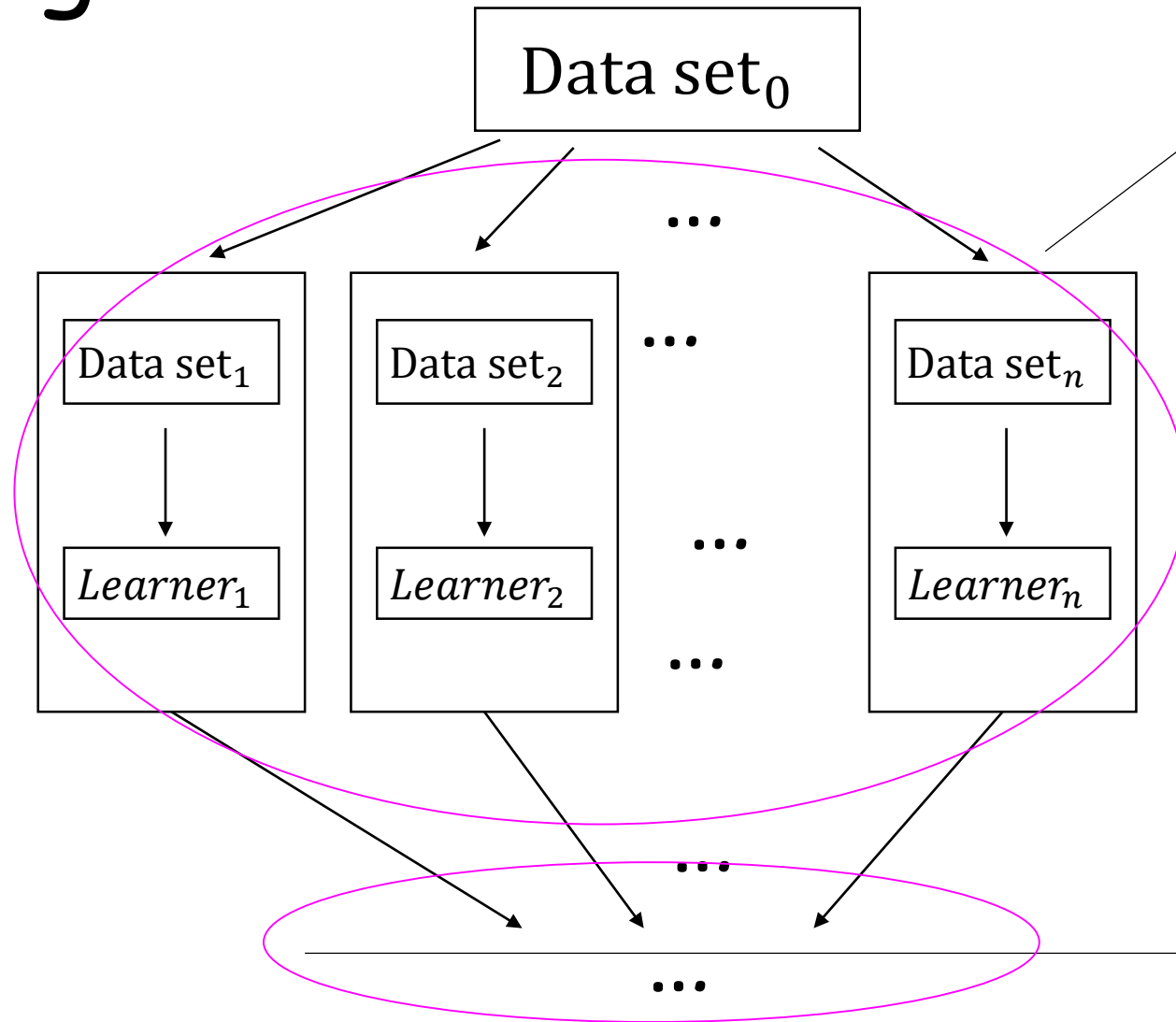
很多成功的集成学习方法

- 序列化方法(个体学习器间存在强依赖关系、必须串行生成的序列化方法)
 - AdaBoost [Freund & Schapire, JCSS97]
 - GradientBoost [Friedman, AnnStat01]
 -
- 并行化方法(个体学习器间不存在强依赖关系、可同时生成的并行化方法)
 - Bagging [Breiman, MLJ06]
 - Random Forest [Breiman, MLJ01]
 - Random Subspace [Ho, TPAMI98]
 -

Boosting



Bagging



Bootstrap a set of learners
generate many data sets from the original data set through bootstrap sampling (random sampling with replacement), then train an individual learner per data set

Voting for classification
The output is the class label receiving the most number of votes

Averaging for regression
The output is the average output of the individual learners

学习器结合

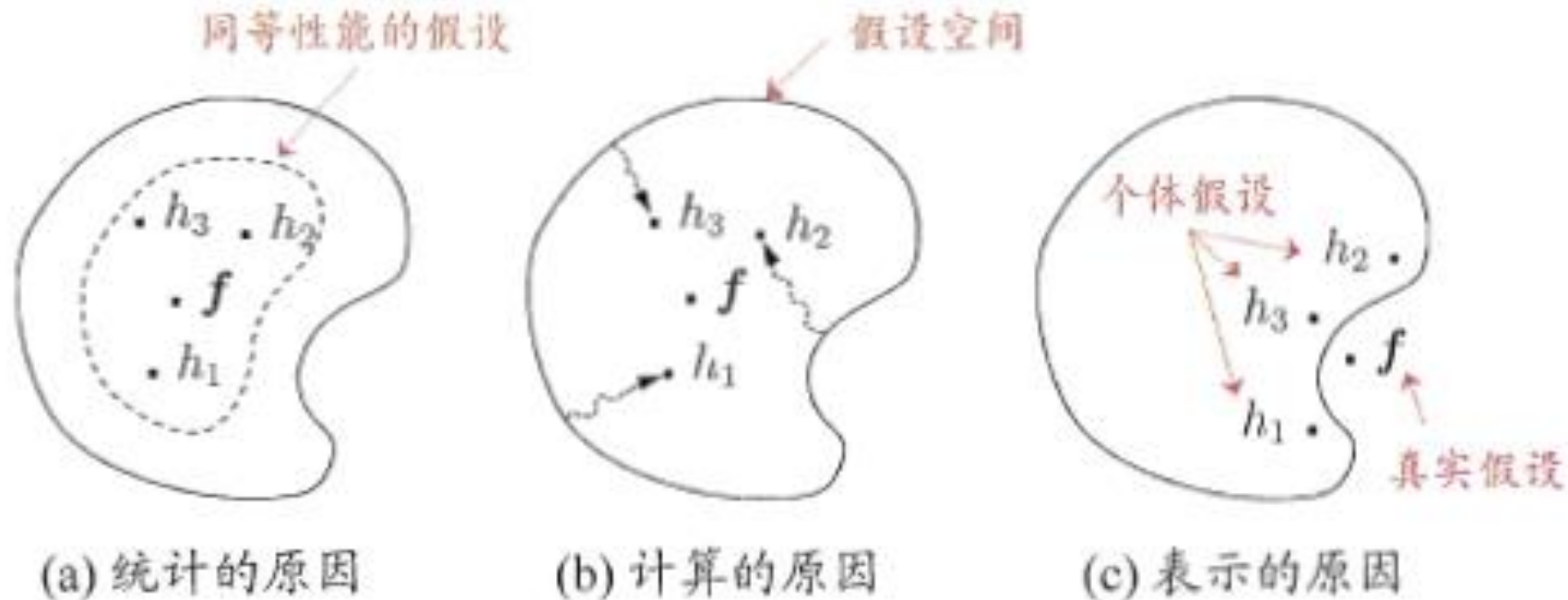


图 8.8 学习器结合可能从三个方面带来好处 [Dietterich, 2000]

对数值型输出 $h_i(\mathbf{x}) \in \mathbb{R}$, 最常见的结合策略是使用平均法 (averaging).

- 简单平均法 (simple averaging)

$$H(\mathbf{x}) = \frac{1}{T} \sum_{i=1}^T h_i(\mathbf{x}) .$$

- 常见结合方法
 - 平均法
 - 简单平均法

学习器结合

- 加权平均法
- 加权平均法(weighted averaging)

$$H(\mathbf{x}) = \sum_{i=1}^T w_i h_i(\mathbf{x}) .$$

其中 w_i 是个体学习器 h_i 的权重, 通常要求 $w_i \geq 0$, $\sum_{i=1}^T w_i = 1$.

- 投票法

- 绝对多数投票法

h_i 在样本 $\mathbf{x}$ 上的预测输出表示为一个 N 维向量 $(h_i^1(\mathbf{x}); h_i^2(\mathbf{x}); \dots; h_i^N(\mathbf{x}))$, 其中 $h_i^j(\mathbf{x})$ 是 h_i 在类别标记 c_j 上的输出.

- 绝对多数投票法(majority voting)

$$H(\mathbf{x}) = \begin{cases} c_j, & \text{if } \sum_{i=1}^T h_i^j(\mathbf{x}) > 0.5 \sum_{k=1}^N \sum_{i=1}^T h_i^k(\mathbf{x}) ; \\ \text{reject}, & \text{otherwise.} \end{cases}$$

即若某标记得票过半数, 则预测为该标记; 否则拒绝预测.

学习器结合

- 相对多数投票法

$$H(\mathbf{x}) = c_{\arg \max_j \sum_{i=1}^T h_i^j(\mathbf{x})}$$

- 加权投票法

$$H(\mathbf{x}) = c_{\arg \max_j \sum_{i=1}^T w_i h_i^j(\mathbf{x})} .$$

与加权平均法类似, w_i 是 h_i 的权重, 通常 $w_i \geq 0$, $\sum_{i=1}^T w_i = 1$

学习器结合

- 学习法:当训练数据很多时, 通过另一个学习器来进行结合, **stacking**是学习法的典型代表。
 - 初级学习器: 个体学习器
 - 次级学习器或元学习器: 用于结合的学习器

Stacking

使用初级学习算法 $\mathcal{L}_t$
产生初级学习器 h_t .

生成次级训练集.

在 $\mathcal{D}'$ 上用次级学习算法 $\mathcal{L}$ 产生次级学习器 h' .

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
初级学习算法 $\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_T$;
次级学习算法 $\mathcal{L}$.

过程:

```
1: for  $t = 1, 2, \dots, T$  do
2:    $h_t = \mathcal{L}_t(D)$ ;
3: end for
4:  $D' = \emptyset$ ;
5: for  $i = 1, 2, \dots, m$  do
6:   for  $t = 1, 2, \dots, T$  do
7:      $z_{it} = h_t(\mathbf{x}_i)$ ;
8:   end for
9:    $D' = D' \cup ((z_{i1}, z_{i2}, \dots, z_{iT}), y_i)$ ;
10: end for
11:  $h' = \mathcal{L}(D')$ ;
输出:  $H(\mathbf{x}) = h'(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x}))$ 
```

图 8.9 Stacking 算法

多样性

- “多样性(diversity)” 是集成学习的关键
误差-分歧分解(error-ambiguity decomposition)

- 多样性度量：度量集成中个体分类器的多样性。

给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, 对二分类任务, $y_i \in \{-1, +1\}$, 分类器 h_i 与 h_j 的预测结果列联表(contingency table)为

	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

其中, a 表示 h_i 与 h_j 均预测为正类的样本数目; b 、 c 、 d 含义由此类推;
 $a + b + c + d = m$.

- 不合度量(disagreement measure)

$$dis_{ij} = \frac{b+c}{m} . \quad (8.37)$$

dis_{ij} 的值域为 $[0, 1]$. 值越大则多样性越大.

- 相关系数(correlation coefficient)

$$\rho_{ij} = \frac{ad - bc}{\sqrt{(a+b)(a+c)(c+d)(b+d)}} . \quad (8.38)$$

ρ_{ij} 的值域为 $[-1, 1]$. 若 h_i 与 h_j 无关, 则值为 0; 若 h_i 与 h_j 正相关则值为正, 否则为负.

- Q -统计量(Q -statistic)

$$Q_{ij} = \frac{ad - bc}{ad + bc} . \quad (8.39)$$

Q_{ij} 与相关系数 ρ_{ij} 的符号相同, 且 $|Q_{ij}| \leq |\rho_{ij}|$.

- κ -统计量(κ -statistic)

$$\kappa = \frac{p_1 - p_2}{1 - p_2} . \quad (8.40)$$

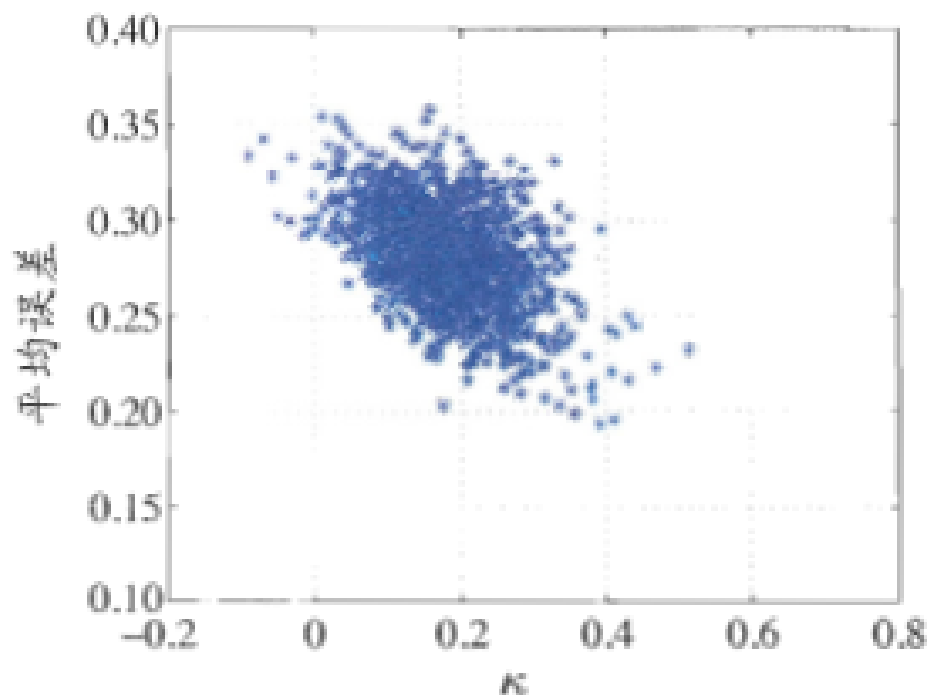
其中, p_1 是两个分类器取得一致的概率; p_2 是两个分类器偶然达成一致的
概率, 它们可由数据集 D 估算:

$$p_1 = \frac{a + d}{m}, \quad (8.41)$$

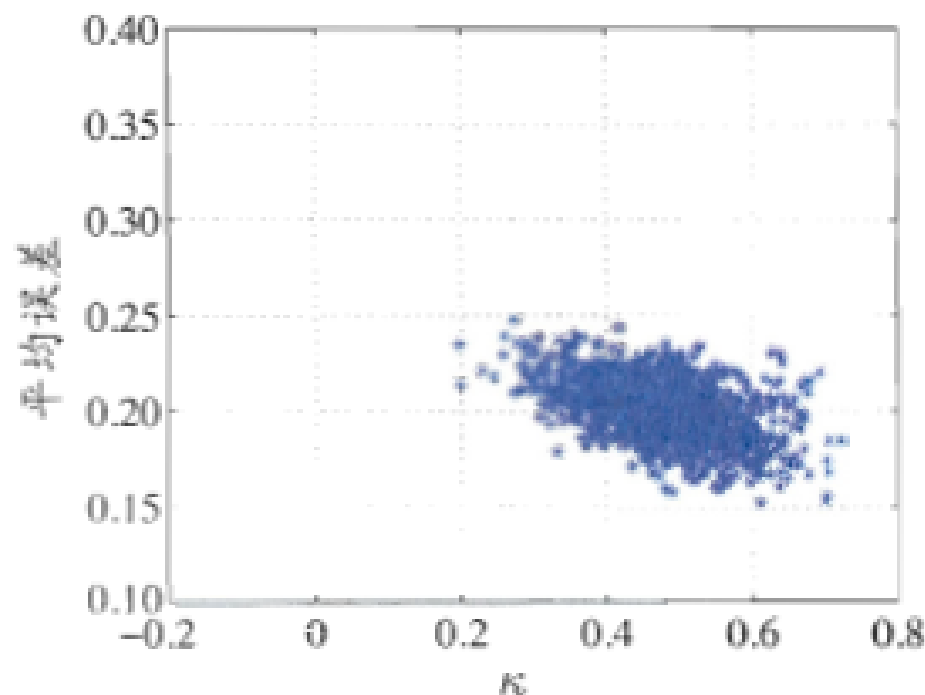
$$p_2 = \frac{(a + b)(a + c) + (c + d)(b + d)}{m^2}. \quad (8.42)$$

若分类器 h_i 与 h_j 在 D 上完全一致, 则 $\kappa = 1$; 若它们仅是偶然达成一致,
则 $\kappa = 0$. κ 通常为非负值, 仅在 h_i 与 h_j 达成一致的
概率甚至低于偶然性的情况下取负值.

例如著名的“ κ -误差图”，就是将每一对分类器作为图上的一个点，横坐标是这对分类器的 κ 值，纵坐标是它们的平均误差，图 8.10 给出了一个例子。显然，数据点云的位置越高，则个体分类器准确性越低；点云的位置越靠右，则个体学习器的多样性越小。



(a) AdaBoost 集成



(b) Bagging 集成

图 8.10 在 UCI 数据集 *tic-tac-toe* 上的 κ -误差图。每个集成含 50 棵 C4.5 决策树

多样性增强

- 数据样本扰动
 - 常基于采样法：**Bagging**自助采样，**AdaBoost**使用序列采样。
 - “不稳定学习器”：训练数据稍有变化，就会导致学习器有显著的变动，如决策树、神经网络等；

多样性增强

- 输入属性扰动

- “稳定学习器”：对数据样本的扰动不敏感，如线性学习器、支持向量机、朴素贝叶斯、**k**近邻居等；
- 从不同子空间训练出的个体学习器必然有所不同。
- 随机子空间算法

d' 小于初始属性数 d .

$\mathcal{F}_t$ 包含 d' 个随机选取的属性, D_t 仅保留 $\mathcal{F}_t$ 中的属性.

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
基学习器数 T ;
子空间属性数 d' .

过程:

```
1: for  $t = 1, 2, \dots, T$  do
2:    $\mathcal{F}_t = \text{RS}(D, d')$ 
3:    $D_t = \text{Map}_{\mathcal{F}_t}(D)$ 
4:    $h_t = \mathcal{L}(D_t)$ 
5: end for
```

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\text{Map}_{\mathcal{F}_t}(\mathbf{x})) = y)$

图 8.11 随机子空间算法

多样性增强

- 输出表示扰动
 - 对输出表示进行操纵以增强多样性。如随机翻转（随机改变一些训练样本的标记）、输出进行转化（对输出表示进行转化，将分类器输出转化为回归输出后构建个体学习器）。
- 算法参数扰动
 - 基学习算法一般都有参数，通过随机设不同的参数，往往可产生差别较大的个体学习器。

Boosting

提升方法 AdaBoost 算法

提升方法的基本思路

AdaBoost 算法

AdaBoost 的例子

提升方法的基本思路

- **1984, Kearns和Valiant**: 强可学习(**strongly learnable**)和弱可学习(**weakly learnable**)
 - 在概率近似正确 (**probably approximately correct, PAC**)学习的框架中, 一个概念 (类), 如果存在一个多项式的学习算法能够学习它, 并且正确率很高, 称这个概念是强可学习的;
 - 一个概念 (类), 如果存在一个多项式的学习算法能够学习它, 学习的正确率仅比随机猜测略好, 则称这个概念是弱可学习的。
- **1989, Schapire**证明: 在**PAC**学习的框架下, 一个概念是强可学习的充分必要条件是这个概念是弱可学习。

问题的提出：

- 只要找到一个比随机猜测略好的弱学习算法就可以直接将其提升为强学习算法，而不必直接去找很难获得的强学习算法。

- 怎样实现弱学习转为强学习？
 - 例如：学习算法**A**在**a**情况下失效，学习算法**B**在**b**情况下失效，那么在**a**情况下可以用**B**算法，在**b**情况下可以用**A**算法解决。这说明通过某种合适的方式把各种算法组合起来，可以提高准确率。
- 为实现弱学习互补，面临两个问题：
 - 怎样获得不同的弱分类器？
 - 怎样组合弱分类器？

怎样获得不同的弱分类器？

- 使用不同的弱学习算法得到不同基本学习器
 - 参数估计、非参数估计...
- 使用相同的弱学习算法，但用不同的参数
 - **K-Mean**不同的**K**，神经网络不同的隐含层...
- 相同输入对象的不同表示凸显事物不同的特征
- 使用不同的训练集
 - 装袋 (**bagging**)
 - 提升 (**boosting**)

- **Bagging**, 也称为自举汇聚法(**bootstrap aggregating**)
 - 从原始数据集选择 S 次后得到 S 个新数据集
 - 新数据集和原数据集的大小相等
 - 每个数据集都是通过原始数据集中随机选择样本来进行替换而得到的。
 - S 个数据集建好之后, 将某个学习算法分别作用于每个数据集就得到 S 个分类器。
 - 选择分类器投票结果中最多的类别作为最后的分类结果。
 - 改进的**Bagging**算法, 如随机森林等。

怎样组合弱分类器？

- 多专家组合

- 并行结构：所有的弱分类器都给出各自的预测结果，通过“组合器”把这些预测结果转换为最终结果。 **eg.**投票（**voting**）及其变种、混合专家模型

- 多级组合

- 串行结构：其中下一个分类器只在前一个分类器预测不够准（不够自信）的实例上进行训练或检测。 **eg.**级联算法（**cascading**）

AdaBoost算法

- 1990年，Schapire最先构造出一种多项式级的算法，即最初的Boost算法；
- 1993年，Drunker和Schapire第一次将神经网络作为弱学习器，应用Boosting算法解决OCR问题；
- 1995年，Freund和Schapire提出了Adaboost (Adaptive Boosting)算法，效率和原来Boosting算法一样，但是不需要任何关于弱学习器性能的先验知识，可以非常容易地应用到实际问题中

AdaBoost算法

$$h_1(x) \in \{-1, +1\}$$

$$h_2(x) \in \{-1, +1\}$$

⋮

$$h_T(x) \in \{-1, +1\}$$

Weak classifiers

$$H_T(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

strong classifier

slightly better than random

- 两个问题如何解决

- 每一轮如何改变训练数据的权值或概率分布？

AdaBoost：提高那些被前一轮弱分类器错误分类样本的权值，降低那些被正确分类样本的权值

- 如何将弱分类器组合成一个强分类器？

AdaBoost：加权多数表决，加大分类误差率小的弱分类器的权值，使其在表决中起较大的作用，减小分类误差率大的弱分类器的权值，使其在表决中起较小的作用。

- 输入：二分类的训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- 输出：最终分类器 $G(x)$

$$x_i \in \mathcal{X} \subseteq \mathbf{R}^n, \quad y_i \in \mathcal{Y} = \{-1, +1\}$$

1) 初始化训练数据的起始权值分布

$$D_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1N}) \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

步骤(1) 假设训练数据集具有均匀的权值分布，即每个训练样本在基本分类器的学习中作用相同，这一假设保证第 1 步能够在原始数据上学习基本分类器 $G_1(x)$ 。

2) 对 m 个弱分类器 $m=1,2, \dots M$

a) 在权值 D_m 下训练数据集，得到弱分类器

步骤(2) AdaBoost 反复学习基本分类器，在每一轮 $m=1,2,\dots,M$ 顺次地执行下列操作：

(a) 使用当前分布 D_m 加权的训练数据集，学习基本分类器 $G_m(x)$.

$$G_m(x): \mathcal{X} \rightarrow \{-1, +1\}$$

b) 计算 G_m 的训练误差

$$e_m = P(G_m(x_i) \neq y_i) = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i)$$

这里， w_{mi} 表示第 m 轮中第 i 个实例的权值， $\sum_{i=1}^N w_{mi} = 1$. 这表明， $G_m(x)$ 在加权的训练数据集上的分类误差率是被 $G_m(x)$ 误分类样本的权值之和，由此可以看出数据权值分布 D_m 与基本分类器 $G_m(x)$ 的分类误差率的关系.

c) 计算 G_m 的系数

$$G_m(x): \mathcal{X} \rightarrow \{-1, +1\}$$

(c) 计算基本分类器 $G_m(x)$ 的系数 α_m . α_m 表示 $G_m(x)$ 在最终分类器中的重要性. 由式 (8.2) 可知, 当 $e_m \leq \frac{1}{2}$ 时, $\alpha_m \geq 0$, 并且 α_m 随着 e_m 的减小而增大, 所以分类误差率越小的基本分类器在最终分类器中的作用越大.

d) 更新训练数据集的权值分布

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

Z 是规范化因子

$$D_{m+1} = (w_{m+1,1}, \dots, w_{m+1,i}, \dots, w_{m+1,N})$$
$$w_{m+1,i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), \quad i = 1, 2, \dots, N$$

$$Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i))$$

(d) 更新训练数据的权值分布为下一轮作准备. 式 (8.4) 可以写成:

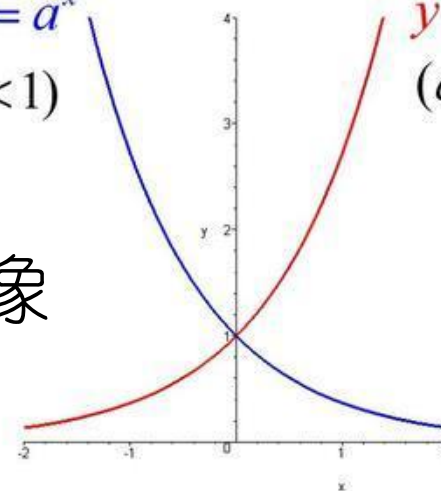
$$w_{m+1,i} = \begin{cases} \frac{w_{mi}}{Z_m} e^{-\alpha_m}, & G_m(x_i) = y_i \\ \frac{w_{mi}}{Z_m} e^{\alpha_m}, & G_m(x_i) \neq y_i \end{cases}$$

由此可知, 被基本分类器 $G_m(x)$ 误分类样本的权值得以扩大, 而被正确分类样本的权值却得以缩小. 两相比较, 误分类样本的权值被放大 $e^{2\alpha_m} = \frac{e_m}{1-e_m}$ 倍. 因此,

误分类样本在下一轮学习中起更大的作用. 不改变所给的训练数据, 而不断改变训练数据权值的分布, 使得训练数据在基本分类器的学习中起不同的作用, 这是 AdaBoost 的一个特点.

$$\begin{aligned} y &= a^x & (0 < a < 1) \\ y &= a^x & (a > 1) \end{aligned}$$

指数函数图像



3) 构建弱分类器的线性组合

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

步骤(3) 线性组合 $f(x)$ 实现 M 个基本分类器的加权表决. 系数 α_m 表示了基本分类器 $G_m(x)$ 的重要性, 这里, 所有 α_m 之和并不为 1. $f(x)$ 的符号决定实例 x 的类, $f(x)$ 的绝对值表示分类的确信度. 利用基本分类器的线性组合构建最终分类器是 AdaBoost 的另一特点.

4) 得到最终分类器:

$$G(x) = \text{sign}(f(x)) = \text{sign}\left(\sum_{m=1}^M \alpha_m G_m(x)\right)$$

例1. AdaBoost的例子

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- 初始化

$$D_1 = (w_{11}, w_{12}, \dots, w_{110})$$
$$w_{1i} = 0.1, \quad i = 1, 2, \dots, 10$$

$m=1$

- a) 在权值分布为 **D1** 的数据集上，阈值取 **2.5**，分类误差率最小，基本弱分类器：

$$G_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

- b) $G_1(x)$ 的误差率：
$$e_1 = P(G_1(x_i) \neq y_i) = 0.3$$

- c) $G_1(x)$ 的系数：
$$\alpha_1 = \frac{1}{2} \log \frac{1 - e_1}{e_1} = 0.4236$$

- d) 更新训练数据的权值分布

$$D_2 = (w_{21}, \dots, w_{2i}, \dots, w_{210})$$

$$w_{2i} = \frac{w_{1i}}{Z_1} \exp(-\alpha_1 y_i G_1(x_i)), \quad i = 1, 2, \dots, 10$$

$$D_2 = (0.0715, 0.0715, 0.0715, 0.0715, 0.0715, 0.0715, \\ 0.1666, 0.1666, 0.1666, 0.0715)$$

$$f_1(x) = 0.4236 G_1(x)$$

- 弱基本分类器 $G(x) = \text{sign}[f_1(x)]$ 在更新的数据集上有3个误分类：点7,8,9

$m=2$

- a) 在权值分布 D_2 上，阈值 $v=8.5$ 时，分类误差率最低

$$G_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x > 8.5 \end{cases}$$

- b) 误差率

$$e_2 = 0.2143.$$

- c) 计算

$$\alpha_2 = 0.6496$$

- d) 更新权值分布

$$D_3 = (0.0455, 0.0455, 0.0455, 0.1667, 0.1667, 0.1667, \\ 0.1060, 0.1060, 0.1060, 0.0455)$$

$$f_2(x) = 0.4236G_1(x) + 0.6496G_2(x)$$

- 分类器 $G(x)=\text{sign}[f_2(x)]$ 有三个误分类点：4,5,6

$m=3$

- a) 在权值分布D3上，阈值 $v=5.5$ 时，分类误差率最低

$$G_3(x) = \begin{cases} 1, & x > 5.5 \\ -1, & x < 5.5 \end{cases}$$

- b) 误差率

$$e_3 = 0.1820.$$

- c) 计算

$$\alpha_3 = 0.7514$$

- d) 更新权值分布

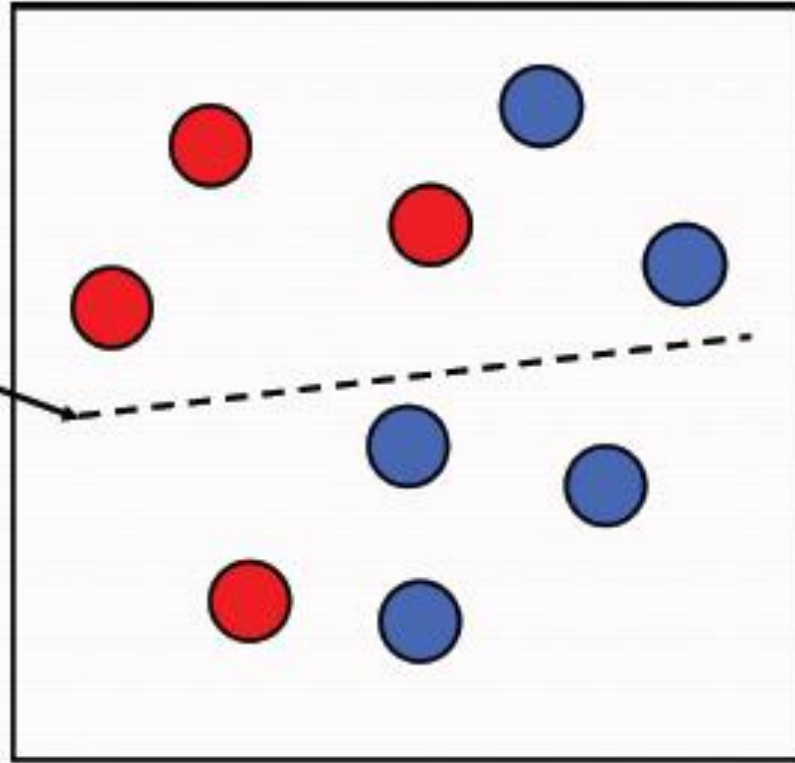
$$D_4 = (0.125, 0.125, 0.125, 0.102, 0.102, 0.102, 0.065, 0.065, 0.065, 0.125)$$

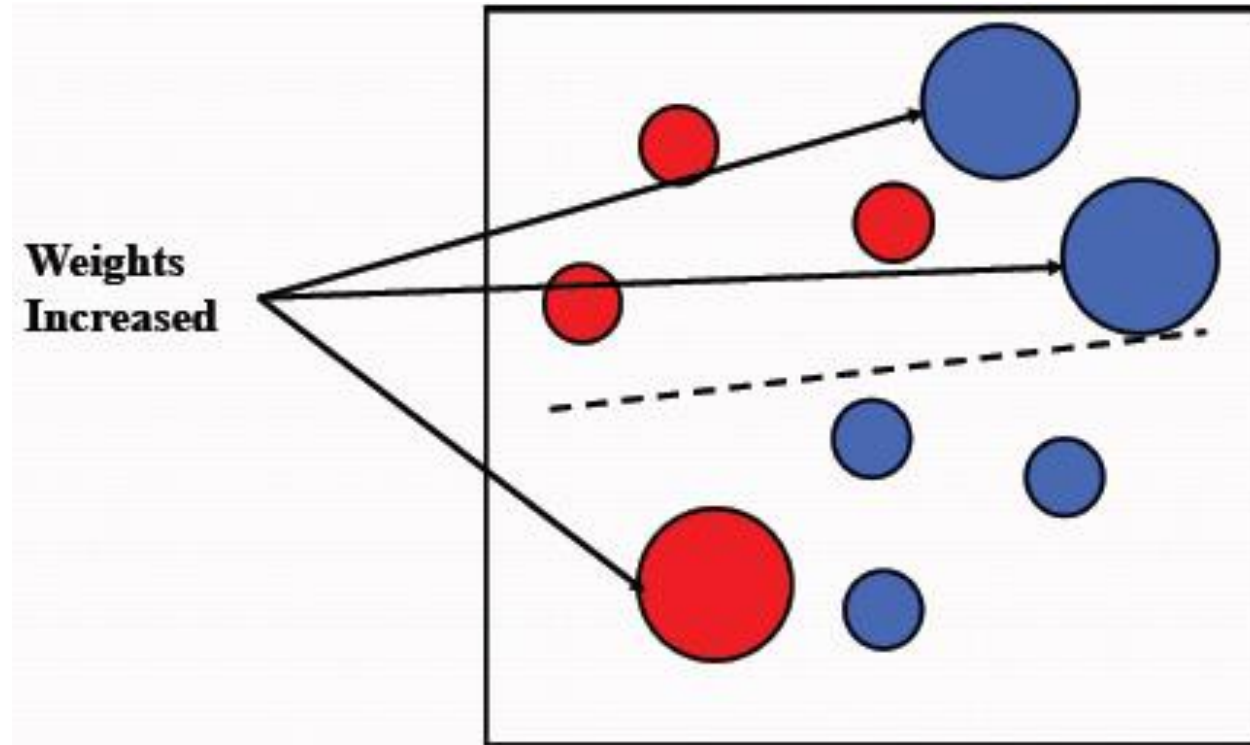
$$f_3(x) = 0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)$$

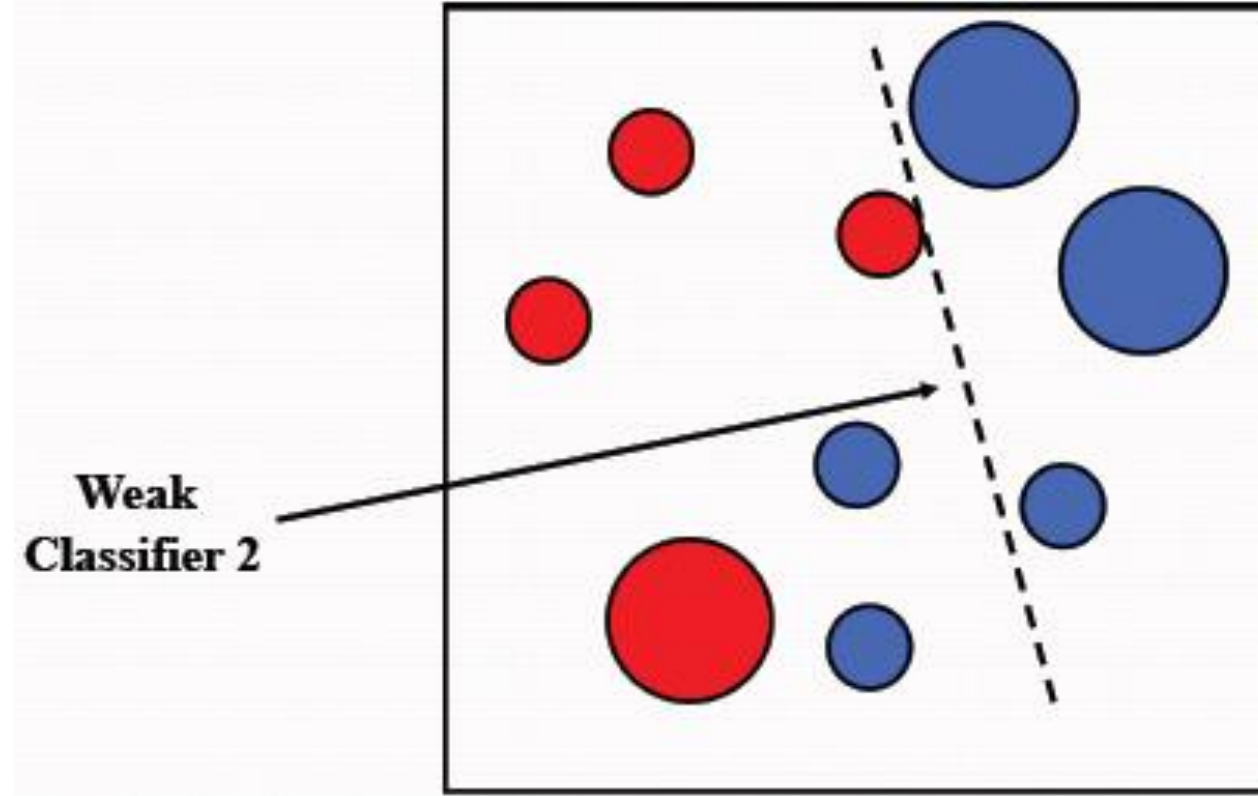
- 分类器 $G(x)=\text{sign}[f_3(x)]$ 没有误分类点

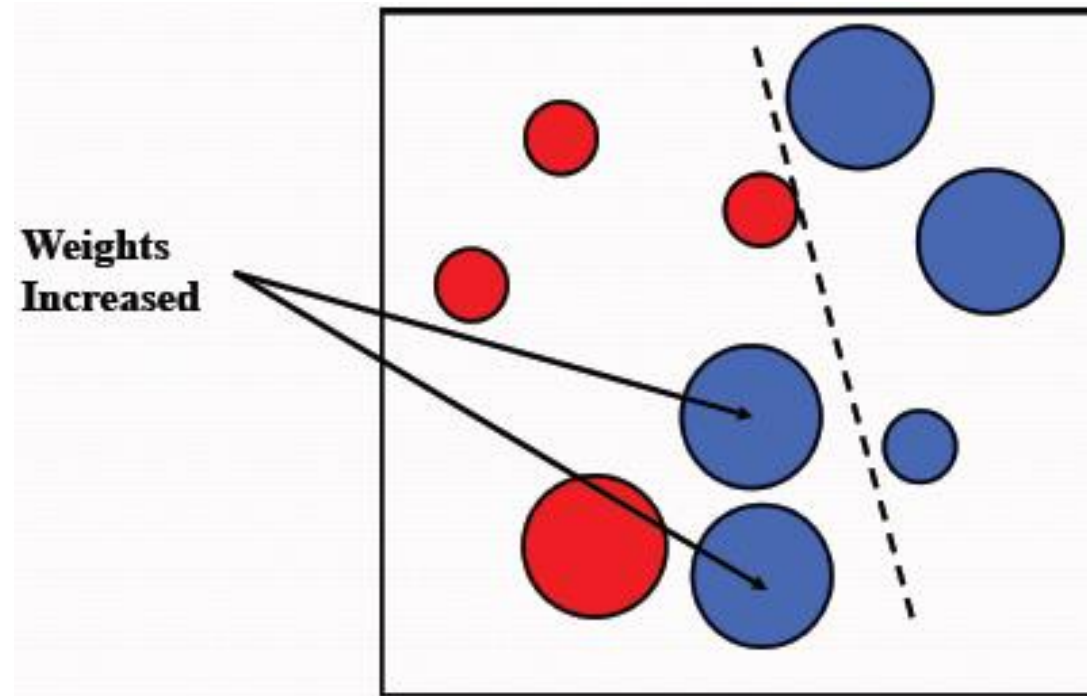
$$G(x) = \text{sign}[f_3(x)] = \text{sign}[0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)]$$

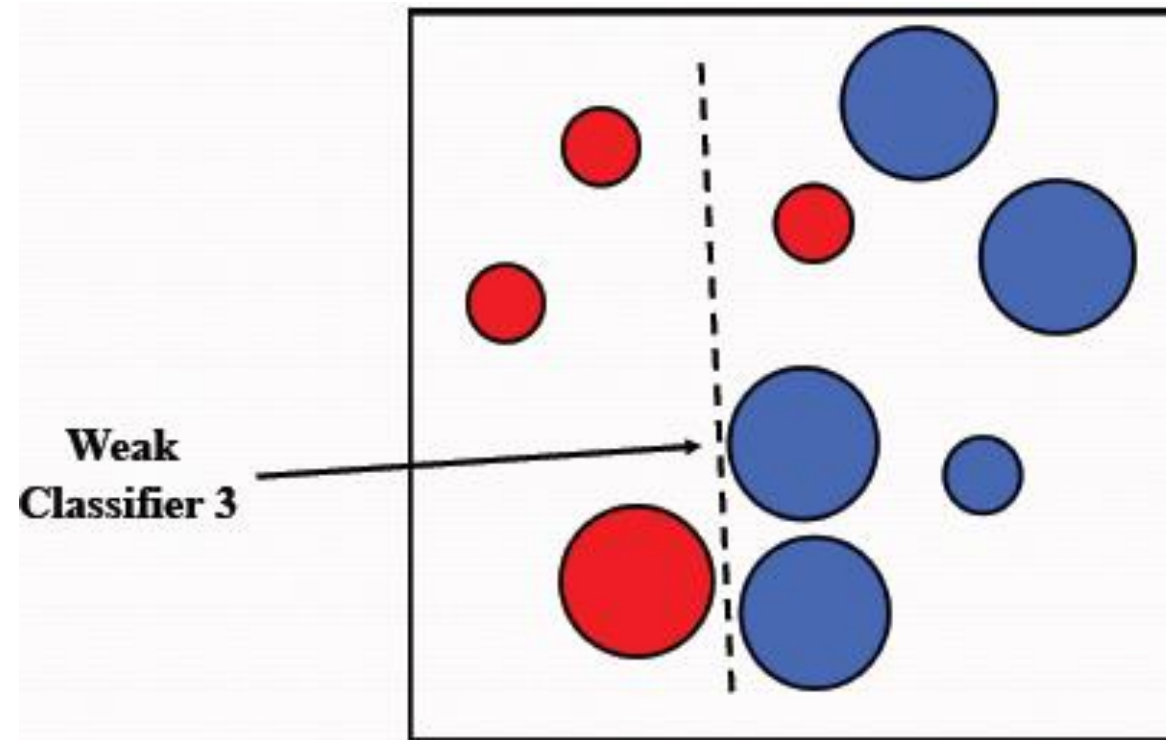
**Weak
Classifier 1**



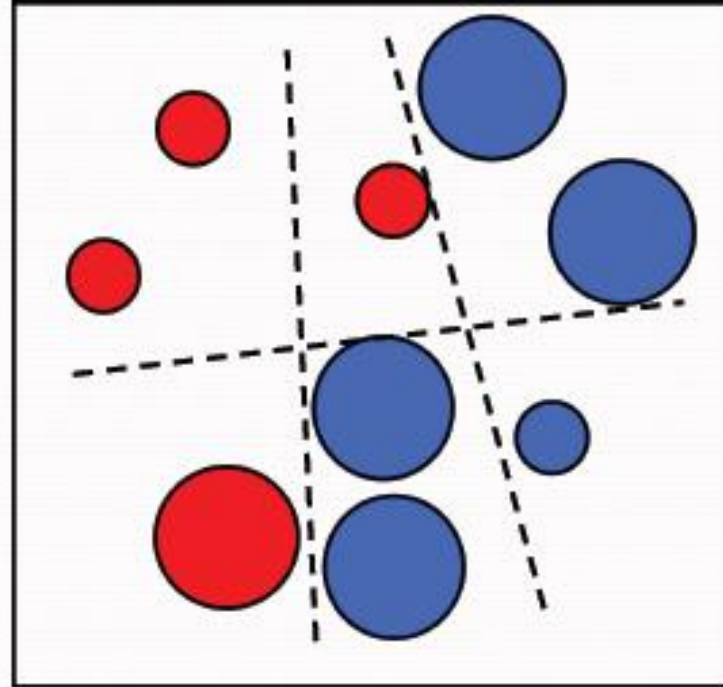








**Final classifier is
a combination of weak
classifiers**



AdaBoost的训练误差分析

• 由

$$\begin{aligned}Z_m &= \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i)) \\f(x) &= \sum_{m=1}^M \alpha_m G_m(x) \\G(x) &= \text{sign}(f(x)) = \text{sign}\left(\sum_{m=1}^M \alpha_m G_m(x)\right)\end{aligned}$$

定理： **AdaBoost**算法最终分类器的训练误差界为：

$$\frac{1}{N} \sum_{i=1}^N I(G(x_i) \neq y_i) \leq \frac{1}{N} \sum_i \exp(-y_i f(x_i)) = \prod_m Z_m$$

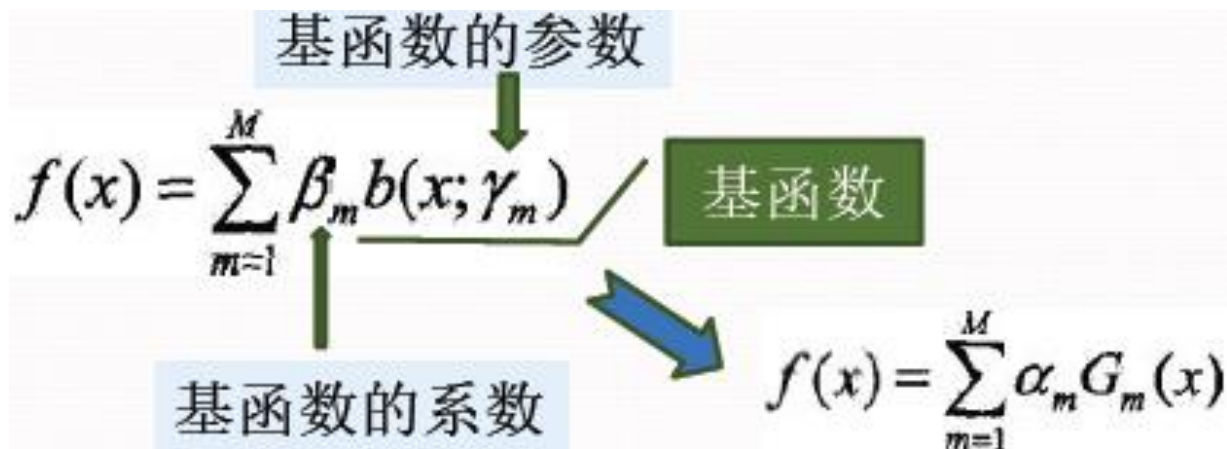
AdaBoost算法的解释

前向分布算法

前向分布算法与AdaBoost

前向分布算法

- 加法模型 (additive model)



- 给定训练数据和损失函数 $L(y, f(x))$, 学习加法模型 $f(x)$ 成为经验风险极小化即损失函数极小化问题

$$\min_{\beta_m, \gamma_m} \sum_{i=1}^N L\left(y_i, \sum_{m=1}^M \beta_m b(x_i; \gamma_m)\right)$$

复杂的优化问题

前向分布算法

- 前向分步算法 **Forward stagewise algorithm** 求解思路：
 - 根据学习的是加法模型，如果能够从前向后，每一步只学习一个基函数及其系数，逐步逼近优化目标函数式，
 - 具体，每步只需优化如下损失函数：

$$\min_{\beta_m, \gamma_m} \sum_{i=1}^N L\left(y_i, \sum_{m=1}^M \beta_m b(x_i; \gamma_m)\right)$$

前向分布算法

• 输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$

损失函数 $L(y, f(x))$ ， 基函数集 $\{b(x, \gamma)\}$

• 输出：加法模型 $f(x)$

• 1 初始化 $f_0(x)=0$

• 2 对 $m=1, 2, \dots, M$

• a、极小化损失函数
得到参数 β_m, γ_m

• b、更新

• 3 得到加法模型

$$(\beta_m, \gamma_m) = \arg \min_{\beta, \gamma} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma))$$

$$f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m)$$

$$f(x) = f_M(x) = \sum_{m=1}^M \beta_m b(x; \gamma_m)$$

前向分布算法与 **AdaBoost**

- 定理：**AdaBoost**算法是前向分步加法算法的特例，模型是由基本分类器组成的加法模型，损失函数是指数函数。

提升树

提升树模型

提升树算法

梯度提升

提升树模型

- 提升树是以分类树或回归树为基本分类器的提升方法；提升树被认为是统计学习中性能最好的方法之一。
- 提升树模型 (**boosting tree**)
 - 提升方法实际采用：加法模型(即基函数的线性组合)与前向分步算法，以决策树为基函数；
 - 对分类问题决策树是二叉分类树，
 - 对回归问题决策树是二叉回归树，
 - 基本分类器 $x < v$ 或 $x > v$ ，可以看作是由一个根结点直接连接两个叶结点的简单决策树，即所谓的决策树桩(**decision stump**)。

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$

$T(x; \Theta_m)$ 表示决策树； Θ_m 为决策树的参数； M 为树的个数。

提升树算法

- 前向分步算法：
- 首先确定初始提升树：
- 第 m 步的模型：

$$f_0(x) = 0$$

$$f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$$

其中， $f_{m-1}(x)$ 为当前模型，通过经验风险极小化确定下一棵决策树的参数 Θ_m

$$\hat{\Theta}_m = \arg \min_{\Theta_m} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m))$$

由于树的线性组合可以很好地拟合训练数据，即使数据中的输入与输出之间的关系很复杂也是如此，所以提升树是一个高功能的学习算法。

- 针对不同问题的提升树学习算法，使用的损失函数不同：
 - 平方误差损失函数的回归问题
 - 指数损失函数的分类问题
 - 一般损失函数的一般决策问题
- 对二类分类问题：提升树算法只需将**AdaBoost**算法中的基本分类器限制为二类分类树即可，这时的提升树算法是**AdaBoost**算法的特殊情况。

- 回归问题提升树

已知训练数据集：

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}, x_i \in \mathcal{X} \subseteq \mathbf{R}^n, y_i \in \mathcal{Y} \subseteq \mathbf{R}$$

$\mathbf{X}$ 为输入空间， $\mathbf{y}$ 为输出空间，

将 $\mathbf{X}$ 划分为 $\mathbf{J}$ 个互不相交的区域 $\mathbf{R}_1, \mathbf{R}_2, \dots, \mathbf{R}_j, \dots, \mathbf{R}_J$,并且在每个区域上确定输出的常量 $\mathbf{c}_j$ ，那么树可表示为：

$$T(x; \Theta) = \sum_{j=1}^J c_j I(x \in R_j)$$

$$\Theta = \{(R_1, c_1), (R_2, c_2), \dots, (R_J, c_J)\}$$

$\mathbf{J}$ 是回归树的复杂度即叶结点个数

- 前向分步算法

$$f_0(x) = 0$$

$$f_m(x) = f_{m-1}(x) + T(x; \Theta_m), \quad m = 1, 2, \dots, M$$

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$

- 在前向分步算法的第 m 步，给定当前 f_{m-1} 需求解

$$\hat{\Theta}_m = \arg \min_{\Theta_m} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m))$$

- 得到第 m 棵树的参数
- 采用平方损失函数时

$$L(y, f(x)) = (y - f(x))^2$$

$$\begin{aligned} L(y, f_{m-1}(x) + T(x; \Theta_m)) \\ &= [y - f_{m-1}(x) - T(x; \Theta_m)]^2 \\ &= [r - T(x; \Theta_m)]^2 \end{aligned}$$

$$r = y - f_{m-1}(x)$$

例2.

- 根据回归问题的提升树算法，求 $f_1(x)$...

x_i	1	2	3	4	5	6	7	8	9	10
y_i	5.56	5.70	5.91	6.40	6.80	7.05	8.90	8.70	9.00	9.05

首先通过以下优化问题：

$$\min_s \left[\min_{c_1} \sum_{x_i \in R_1} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2} (y_i - c_2)^2 \right]$$

求解训练数据的切分点 s ：

$$R_1 = \{x \mid x \leq s\}, \quad R_2 = \{x \mid x > s\}$$

容易求得在 R_1, R_2 内部使平方损失误差达到最小值的 c_1, c_2 为

$$c_1 = \frac{1}{N_1} \sum_{x_i \in R_1} y_i, \quad c_2 = \frac{1}{N_2} \sum_{x_i \in R_2} y_i$$

这里 N_1, N_2 是 R_1, R_2 的样本点数。

求训练数据的切分点. 根据所给数据, 考虑如下切分点:

1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5

对各切分点, 不难求出相应的 R_1, R_2, c_1, c_2 及

$$m(s) = \min_{c_1} \sum_{x_i \in R_1} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2} (y_i - c_2)^2$$

例如, 当 $s = 1.5$ 时, $R_1 = \{1\}$, $R_2 = \{2, 3, \dots, 10\}$, $c_1 = 5.56$, $c_2 = 7.50$,

$$m(s) = \min_{c_1} \sum_{x_i \in R_1} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2} (y_i - c_2)^2 = 0 + 15.72 = 15.72$$

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
$m(s)$	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

当 $s = 6.5$ 时 $m(s)$ 达到最小值, 此时 $R_1 = \{1, 2, \dots, 6\}$, $R_2 = \{7, 8, 9, 10\}$, $c_1 = 6.24$, $c_2 = 8.91$, 所以回归树 $T_1(x)$ 为

$$T_1(x) = \begin{cases} 6.24, & x < 6.5 \\ 8.91, & x \geq 6.5 \end{cases}$$

$$f_1(x) = T_1(x)$$

表中 $r_{2i} = y_i - f_1(x_i)$, $i = 1, 2, \dots, 10$.

残差表

x_i	1	2	3	4	5	6	7	8	9	10
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

用 $f_1(x)$ 拟合训练数据的平方损失误差:

$$L(y, f_1(x)) = \sum_{i=1}^{10} (y_i - f_1(x_i))^2 = 1.93$$

第 2 步求 $T_2(x)$. 方法与求 $T_1(x)$ 一样, 只是拟合的数据是表 中 的残差, 可以得到:

$$T_2(x) = \begin{cases} -0.52, & x < 3.5 \\ 0.22, & x \geq 3.5 \end{cases}$$

$$f_2(x) = f_1(x) + T_2(x) = \begin{cases} 5.72, & x < 3.5 \\ 6.46, & 3.5 \leq x < 6.5 \\ 9.13, & x \geq 6.5 \end{cases}$$

用 $f_2(x)$ 拟合训练数据的平方损失误差是

$$L(y, f_2(x)) = \sum_{i=1}^{10} (y_i - f_2(x_i))^2 = 0.79$$

$$T_3(x) = \begin{cases} 0.15, & x < 6.5 \\ -0.22, & x \geq 6.5 \end{cases} \quad L(y, f_3(x)) = 0.47 ,$$

$$T_4(x) = \begin{cases} -0.16, & x < 4.5 \\ 0.11, & x \geq 4.5 \end{cases} \quad L(y, f_4(x)) = 0.30 ,$$

$$T_5(x) = \begin{cases} 0.07, & x < 6.5 \\ -0.11, & x \geq 6.5 \end{cases} \quad L(y, f_5(x)) = 0.23 ,$$

$$T_6(x) = \begin{cases} -0.15, & x < 2.5 \\ 0.04, & x \geq 2.5 \end{cases}$$

$$f_6(x) = f_5(x) + T_6(x) = T_1(x) + \cdots + T_5(x) + T_6(x)$$

$$= \begin{cases} 5.63, & x < 2.5 \\ 5.82, & 2.5 \leq x < 3.5 \\ 6.56, & 3.5 \leq x < 4.5 \\ 6.83, & 4.5 \leq x < 6.5 \\ 8.95, & x \geq 6.5 \end{cases}$$

用 $f_6(x)$ 拟合训练数据的平方损失误差是

$$L(y, f_6(x)) = \sum_{i=1}^{10} (y_i - f_6(x_i))^2 = 0.17$$

假设此时已满足误差要求，那么 $f(x) = f_6(x)$ 即为所求提升树。

梯度提升

- 提升树利用加法模型与前向分步算法实现学习的优化过程，当损失函数是平方损失和指数损失函数时，每一步优化是很简单的，但对一般损失函数而言，往往每一步优化并不容易。
- 针对这一问题，**Freidmao**提出了梯度提升(**gradient boosting**)算法.这是利用最速下降法的近似方法，其关键是利用损失函数的负梯度在当前模型的值

作为回归问题提升树算法中残差的近似值，拟合一个回归树。

$$-\left[\frac{\partial L(y, f(x_i))}{\partial f(x_i)}\right]_{f(x)=f_{m-1}(x)}$$

梯度提升算法

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $x_i \in \mathcal{X} \subseteq \mathbf{R}^n$, $y_i \in \mathcal{Y} \subseteq \mathbf{R}$;

损失函数 $L(y, f(x))$;

输出: 回归树 $\hat{f}(x)$.

(1) 初始化

$$f_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$$

(2) 对 $m = 1, 2, \dots, M$

(a) 对 $i = 1, 2, \dots, N$, 计算

$$r_{mi} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x) = f_{m-1}(x)}$$

(b) 对 r_{mi} 拟合一个回归树, 得到第 m 棵树的叶结点区域 R_{mj} , $j = 1, 2, \dots, J$

(c) 对 $j = 1, 2, \dots, J$, 计算

$$c_{mj} = \arg \min_c \sum_{x_i \in R_{mj}} L(y_i, f_{m-1}(x_i) + c)$$

(d) 更新 $f_m(x) = f_{m-1}(x) + \sum_{j=1}^J c_{mj} I(x \in R_{mj})$

(3) 得到回归树

$$\hat{f}(x) = f_M(x) = \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x \in R_{mj})$$

Homework

2 比较支持向量机、AdaBoost、逻辑斯谛回归模型的学习策略与算法.

8.3 从网上下载或自己编程实现 AdaBoost, 以不剪枝决策树为基学习器, 在西瓜数据集 3.0 α 上训练一个 AdaBoost 集成, 并与图 8.4 进行比较.

西瓜数据集 3.0 α 见 p.89

表 4.5.

西瓜数据集 3.0 α 中密度、含糖率任选一列数据

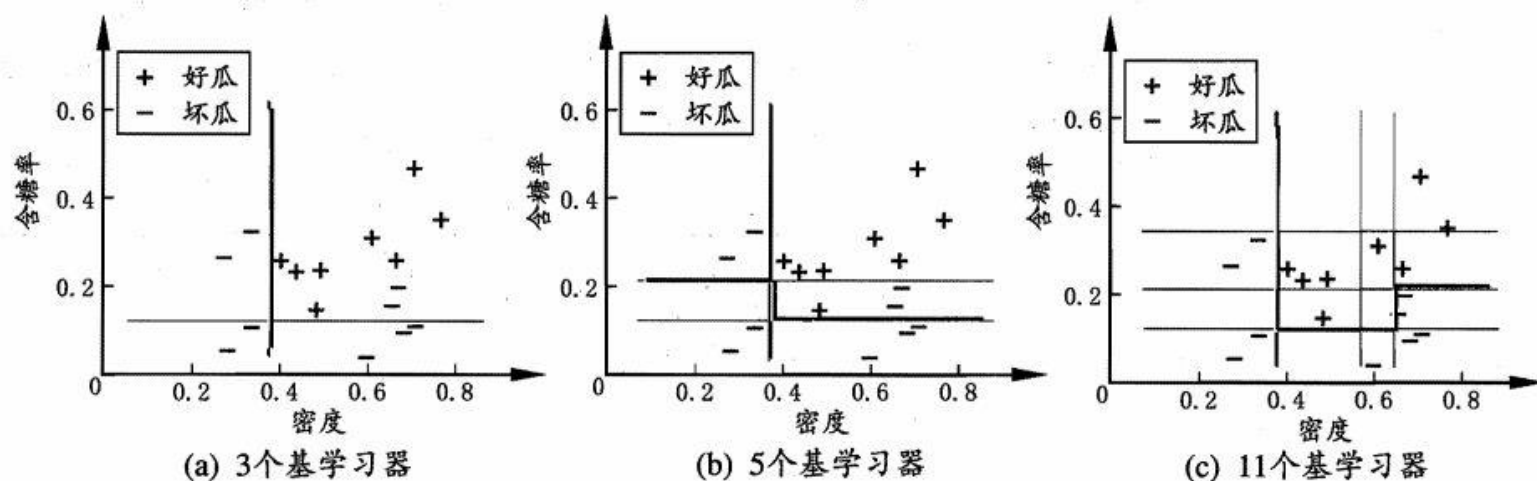


图 8.4 西瓜数据集 3.0 α 上 AdaBoost 集成规模为 3、5、11 时, 集成(红色)与基学习器(黑色)的分类边界.

谢谢！